BrainGrow

Retrieval Indexing & Active Knowledge Maintenance — Spec Addendum

Version 1.0 | March 2026 | Vektas Solutions

Author: Matthew Monroe

---

1. Overview

This addendum specifies two major enhancements to the BrainGrow architecture:

- **Part 1 — FAISS Indexing:** Approximate nearest neighbor indexing for fast retrieval at scale. Replaces brute-force cosine similarity search with GPU-accelerated index lookup. Enables millisecond retrieval at millions of slots.
- **Part 2 — Active Knowledge Maintenance:** A feedback loop that makes the knowledge space self-improving through use. Reactive updating generates targeted negative examples when boundary violations are detected. Proactive auditing identifies high hallucination risk zones before they cause problems.

> *Together these enhancements transform BrainGrow from a static knowledge store into a self-improving developmental system — one that gets more accurate through use rather than degrading over time.*

Part 1 — FAISS Retrieval Indexing

2. Why Brute Force Search Fails at Scale

Current BrainGrow routing computes cosine similarity between the query vector and every active slot on every query. This is O(n) — linear in the number of active slots.

Active Slots	Brute Force Query Time
`10,000`	< 1ms — imperceptible
`200,000`	~10ms — noticeable
`1,000,000`	~50ms — slow
`10,000,000`	~500ms — unusable

FAISS reduces query time to approximately O(log n) regardless of slot count, with GPU acceleration bringing even million-slot queries to under 1ms on the RTX 4070.

3. FAISS Index Types

Index Type	Best For
`IndexFlatIP`	Exact search, small-medium scale (< 500k slots). No approximation. Use during development.

IndexIVFFlat	Large scale (500k-10M slots). Clusters vectors into cells, searches only nearby cells. Slight accuracy tradeoff for large speed gain.
IndexHNSWFlat	Production scale. Hierarchical graph structure. Fastest queries, highest memory use.
IndexIVFPQ	Massive scale with memory constraints. Product quantization compresses vectors. Some accuracy loss.

Recommendation for BrainGrow: Start with IndexFlatIP for development and TinyStories experiments. Move to IndexIVFFlat when slot count exceeds 500k.

4. FAISS Integration — vector_space.py

Add FAISS index as a parallel structure alongside the existing slots tensor. Index is rebuilt after each ingestion stage and updated incrementally during slot assignment.

4.1 Initialization

```
import faiss
import numpy as np

class VectorSpace:
    def __init__(self, n_slots=200_000, dimensions=384):
        self.n_slots = n_slots
        self.dimensions = dimensions
        self.slots = torch.zeros(n_slots, dimensions)
        self.activation = torch.zeros(n_slots)
        self.slot_labels = {}
        self.n_active = 0
        self.domain_registry = {}
        self.stage_number = 0

        # FAISS index — GPU accelerated
        res = faiss.StandardGpuResources()
        flat_index = faiss.IndexFlatIP(dimensions)
        self.index = faiss.index_cpu_to_gpu(res, 0, flat_index)
        self.index_slot_map = []  # maps FAISS position to slot index
```

4.2 Slot Assignment with Index Update

```
def assign_slot(self, embedding: np.ndarray, label: str) -> dict:
    # Find nearest dormant slot
    slot_idx = self._find_dormant_slot(embedding)

    # Store embedding and activate slot
    self.slots[slot_idx] = torch.tensor(embedding)
    self.activation[slot_idx] = 0.5
    self.slot_labels[slot_idx] = label
    self.n_active += 1

    # Add to FAISS index
    vec = embedding.reshape(1, -1).astype('float32')
    faiss.normalize_L2(vec)
    self.index.add(vec)
    self.index_slot_map.append(slot_idx)
```

```
        # Update domain registry
        self.domain_registry.setdefault(label, []).append(slot_idx)

        return {'slot_idx': slot_idx, 'label': label}
```

4.3 FAISS-Accelerated Query Routing — query_router.py

```
    def route_query(self, text: str, top_k: int = 5) -> dict:
        # Encode query
        embedding = model.encode([text], device='cuda')
        vec = embedding.astype('float32')
        faiss.normalize_L2(vec)

        # FAISS search — millisecond speed at any scale
        similarities, faiss_indices = self.index.search(vec, top_k)

        # Map FAISS positions back to slot indices
        matches = []
        for sim, fi in zip(similarities[0], faiss_indices[0]):
            if fi == -1:
                continue
            slot_idx = self.index_slot_map[fi]
            label = self.slot_labels[slot_idx]
            matches.append({
                'slot_idx': slot_idx,
                'label': label,
                'similarity': float(sim),
                'activation': float(self.activation[slot_idx])
            })
            # Reinforce matched slot
            self.activation[slot_idx] = min(
                1.0, self.activation[slot_idx] + 0.1
            )

        return {
            'matches': matches,
            'active_count': self.n_active,
            'dormant_count': self.n_slots - self.n_active
        }
```

4.4 FAISS Save and Load

FAISS index must be saved and loaded alongside the VectorSpace state. Update the save/load methods from the persistence addendum:

```
    def save(self, path: str, description: str = ''):
        # Save PyTorch state as before
        torch.save(state, path)

        # Save FAISS index alongside
        faiss_path = path.replace('.bgstate', '.faiss')
        cpu_index = faiss.index_gpu_to_cpu(self.index)
        faiss.write_index(cpu_index, faiss_path)

    @classmethod
    def load(cls, path: str):
```

```
        vs, meta = super().load(path)

        # Reload FAISS index to GPU
        faiss_path = path.replace('.bgstate', '.faiss')
        cpu_index = faiss.read_index(faiss_path)
        res = faiss.StandardGpuResources()
        vs.index = faiss.index_cpu_to_gpu(res, 0, cpu_index)

        return vs, meta
```

5. Installation

```
# CPU version
pip install faiss-cpu --break-system-packages

# GPU version (recommended for RTX 4070)
pip install faiss-gpu --break-system-packages
```

Part 2 — Active Knowledge Maintenance

6. The Core Insight

The quantum fermentation experiment demonstrated a critical finding: as a knowledge domain grows richer with positive examples, plausible-sounding false combinations become harder to detect — not easier. The positive/negative knowledge ratio is the primary predictor of hallucination risk.

> *A neural network — biological or artificial — fed rich information about a concept without counterbalancing negative knowledge will construct plausible false beliefs with high confidence. The richer the positive knowledge, the more convincing the hallucination.*

Active knowledge maintenance addresses this through two mechanisms: reactive updating when violations are detected, and proactive auditing to identify risk zones before they cause problems.

7. New File — knowledge_maintenance.py

7.1 Reactive Updating

When a boundary violation is detected during a query, automatically generate and ingest a targeted negative example for that specific false combination. Every near-miss makes the system smarter.

```
class ReactiveUpdater:
    def __init__(self, vector_space, growth_engine):
        self.vs = vector_space
        self.ge = growth_engine
        self.violation_log = []

    def handle_violation(self, query: str,
                         nearest_concept: str,
                         domain: str) -> dict:

        # Generate targeted negative example
        negative_text = (
            f'{query} is not a valid concept. '
            f'The components of this query belong to '
            f'unrelated domains with no known intersection.'
```

```
        )
        neg_label = f'{domain}-negative-auto'

        # Validate before ingesting
        # Do not ingest if similarity to existing
        # negative examples is already above 0.85
        # — avoids redundant negative examples
        existing = self.vs.route_query(negative_text, top_k=1)
        if existing['matches'] and \
           existing['matches'][0]['similarity'] > 0.85 and \
           'negative' in existing['matches'][0]['label']:
            return {'action': 'skipped', 'reason': 'already covered'}

        # Ingest negative example
        embedding = model.encode([negative_text], device='cuda')[0]
        self.vs.assign_slot(embedding, neg_label)

        # Log violation
        self.violation_log.append({
            'query': query,
            'domain': domain,
            'negative_label': neg_label,
            'timestamp': datetime.now().isoformat()
        })

        return {'action': 'ingested', 'label': neg_label}
```

7.2 Proactive Risk Auditing

Periodically scan the active slot space and identify domains where positive examples vastly outnumber negative examples. Flag these as hallucination risk zones before they cause problems.

```
def audit_knowledge_balance(vector_space,
                            threshold: float = 10.0) -> list:
    risk_zones = []

    for domain in vector_space.domain_registry:
        if 'negative' in domain:
            continue

        positive_count = len(
            vector_space.domain_registry[domain]
        )
        neg_domain = f'{domain}-negative'
        negative_count = len(
            vector_space.domain_registry.get(neg_domain, [])
        )
        auto_neg = f'{domain}-negative-auto'
        auto_count = len(
            vector_space.domain_registry.get(auto_neg, [])
        )
        total_negative = negative_count + auto_count
        ratio = positive_count / max(total_negative, 1)

        if ratio > threshold:
            risk_zones.append({
```

```
            'domain': domain,
            'positive': positive_count,
            'negative': total_negative,
            'ratio': round(ratio, 1),
            'risk': 'HIGH' if ratio > 20 else 'MEDIUM'
        })

    return sorted(
        risk_zones,
        key=lambda x: x['ratio'],
        reverse=True
    )
```

8. Tab 5 — Network Tab UI Additions

Add the following elements to the existing Network tab in main.py alongside the save/load controls:

- **Run Audit button:** Triggers audit_knowledge_balance() and displays results as a sortable risk table showing domain, positive count, negative count, ratio, and risk level.
- **Risk threshold slider:** Adjustable ratio threshold (default 10.0) above which domains are flagged as risk zones.
- **Auto-update toggle:** When enabled, ReactiveUpdater.handle_violation() fires automatically on every boundary violation detected in Tab 4.
- **Violation log panel:** Shows recent auto-generated negative examples with query, domain, and timestamp. Lets user review and optionally delete auto-generated negatives they disagree with.
- **Manual negative input:** Text input for user to manually add negative examples to any domain without going through Tab 1. Bypasses the chunking pipeline for quick counterbalancing.

---

9. Validation Layer for Auto-Generated Negatives

Auto-generated negative examples carry risk — a poorly formed negative example could itself introduce confusion. The validation layer prevents bad negatives from entering the space.

Validation Check	Action if Failed
`Similarity to existing negatives > 0.85`	Skip — already covered by existing negative example
`Similarity to positive examples in same domain > 0.90`	Flag for human review — negative too similar to positive knowledge
`Query string length < 10 characters`	Skip — query too short to generate meaningful negative
`Auto-negative count for domain > 50`	Pause auto-generation — flag domain for manual review

> *Auto-generated negatives are a first line of defense, not a complete solution. Human review of the violation log should be part of any serious deployment. The system flags, the human decides.*

---

10. Updated File Structure

```
braingrow/
  ├── main.py                  # Updated Tab 5 with audit + violation log
  ├── vector_space.py          # FAISS index integrated
  ├── growth_engine.py         # Batch encoding unchanged
  ├── query_router.py          # FAISS-accelerated routing
  ├── visualizer.py            # Unchanged
  ├── comparison_harness.py    # Unchanged
  ├── knowledge_maintenance.py # NEW — ReactiveUpdater + audit
  ├── requirements.txt         # Add faiss-gpu
  └── saves/
       ├── network.bgstate
       └── network.faiss          # NEW — FAISS index saved alongside
```

---

11. Performance Targets

Operation	Target Performance (RTX 4070)
`Query routing — 200k slots`	< 1ms with FAISS GPU index
`Query routing — 1M slots`	< 5ms with IndexIVFFlat
`Slot assignment + index update`	< 10ms per slot
`Batch encoding — 1k chunks`	< 5 seconds
`Risk audit — 200k slots`	< 2 seconds
`Auto-negative generation`	< 500ms per violation
`FAISS index save`	< 5 seconds for 200k slot index
`FAISS index load to GPU`	< 3 seconds

---

12. Research Significance

The combination of FAISS indexing and active knowledge maintenance moves BrainGrow from proof-of-concept to a credible research platform with the following novel properties:

- **Self-improving accuracy:** The system gets better at catching hallucinations through use — each boundary violation adds a targeted negative example that prevents the same false combination from being missed again.
- **Measurable epistemic health:** The positive/negative ratio audit provides a quantifiable metric for knowledge quality that has no equivalent in standard LLM architectures. A domain with ratio 50:1 is provably more hallucination-prone than one with ratio 5:1.
- **Transparent knowledge gaps:** The audit dashboard makes ignorance visible — not just what the system knows, but where it is most likely to confabulate. Standard LLMs have no equivalent transparency.

- **Scalable to production:** FAISS GPU indexing means the architecture is not limited to toy-scale demonstrations. The same mechanism that works at 10k slots works at 10M slots with millisecond retrieval.

A system that knows what it doesn't know, measures its own hallucination risk, and improves through experience rather than retraining is not just a research curiosity. It is a fundamentally different approach to AI reliability — one that current architectures cannot achieve without this kind of structural foundation.